



Algoritmi e programmazione

AP2022_III_Programmazione [AULA]



OSKAR BICH
226421

Iniziato lunedì, 27 giugno 2022, 11:53

Terminato lunedì, 27 giugno 2022, 13:28

Tempo impiegato 1 ora 35 min.

Informazione

PER ENTRAMBE LE PROVE DI PROGRAMMAZIONE (18 o 12 punti)

- se non indicato diversamente, è consentito utilizzare chiamate a funzioni standard, quali ordinamento per vettori, funzioni su FIFO, LIFO, liste, BST, tabelle di hash, grafi e altre strutture dati, considerate come librerie esterne;
- gli header file riferiti dal codice dovranno essere inclusi nella versione del programma allegata alla relazione;
- i modelli delle funzioni ricorsive **non** sono considerati funzioni standard;
- consegna delle relazioni, per entrambe le tipologie di prova di programmazione: entro GIOVEDÌ 30/06/2022, alle ore 23:59, mediante caricamento su Portale;
- le istruzioni per il caricamento sono pubblicate sul Portale nella sezione Materiale;
- **QUALORA IL CODICE CARICATO CON LA RELAZIONE NON COMPILI CORRETTAMENTE, VERRÀ APPLICATA UNA PENALIZZAZIONE.** Si ricorda che la valutazione del compito viene fatta, senza la presenza del candidato, sulla base dell'elaborato svolto durante l'appello. Non verranno corretti i compiti di cui non sarà stata inviata la relazione nei tempi stabiliti.

Attenzione!

Indicare quale tipologia di esame si intenda svolgere rispondendo alla domanda a scelta multipla seguente (domanda 1). Si noti che è possibile leggere tutte o parte delle domande prima di effettuare la scelta e rispondere alla domanda 1.

Le domande dalla 2 alla 6 sono dedicate all'esame semplificato (traccia da 12pt).

In particolare:

- la domanda 2 fa parte dell'esercizio da 2 punti.
- la domanda 3 fa parte dell'esercizio da 4 punti.
- le domande 4,5,6 fanno parte dell'esercizio da 6 punti.

Le domande dalla 7 in poi sono dedicate all'esame completo (traccia da 18pt).

Un apposito separatore fa da intervallo tra le domande del compito da 12pt e le domande del compito da 18pt.

Domanda 1

Completo

Non valutata

Indicare quale tipologia di esame si intende svolgere.

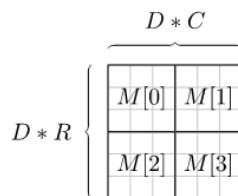
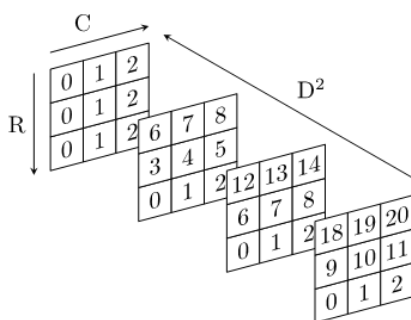
- ☐ (a) 12 Punti - Semplificato
- ☒ (b) 18 Punti - Completo

Domanda 2

Completo

Punteggio max.: 2,00

Una funzione riceve una matrice tridimensionale M di dimensioni $(D^2) \times R \times C$. La funzione `flatten` alloca una nuova matrice bidimensionale di dimensioni $(D \times R) \times (D \times C)$, ricopia i contenuti della matrice ricevuta come argomento con lo schema illustrato a seguire e ritorna la nuova matrice. In sostanza, nella matrice risultante ogni $M[i]$ -esimo livello viene riposizionato "per righe". Nell'esempio seguente D vale 2.



18	19	20	12	13	14
9	10	11	6	7	8
0	1	2	0	1	2
6	7	8	0	1	2
3	4	5	0	1	2
0	1	2	0	1	2

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

```
... flatten(...) {  
  
}
```

Domanda 3

Completo

Punteggio max.: 4,00

Sia dato un albero di grado N, i cui nodi sono definiti dalla seguente struttura C:

```
struct node {  
    char *key;  
    struct node *children[N];  
};
```

Definire la struttura wrapper dell'albero nTREE come ADT di prima categoria evidenziando la suddivisione tra file dei vari contenuti. Si realizzi una funzione wrapper `int countIf(nTREE t);` e la corrispondente funzione di visita ricorsiva, che visiti l'albero e ritorni il conteggio del numero di nodi aventi grado (numero di figli) maggiore del grado del rispettivo nodo padre. Il nodo radice, che non ha un padre, conta 1 per default.

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

NT.h

```
//NT.h  
//scrivere qui il codice
```

NT.c

```
//NT.c  
//scrivere qui il codice
```

```
//scrivere qui il codice delle funzioni richieste  
int countIf(nTREE t) {}
```

Domanda 4

Completo

Punteggio max.: 6,00

Si vogliono generare tutte le sequenze alfabetiche di k (parametro del programma) caratteri che rispettino i seguenti vincoli:

- si possono usare solo lettere minuscole (da 'a' a 'z') e maiuscole (da 'A' a 'Z')
- al massimo la metà dei caratteri possono essere lettere minuscole
- lo stesso carattere, senza distinguere tra maiuscolo o minuscolo, non può apparire più di p (parametro del programma) volte consecutivamente.

Si scriva una funzione ricorsiva in C in grado di generare tutte le sequenze secondo le regole di cui sopra, stampando solo quelle accettabili.

NOTA BENE: seguono altre due domande relative a questo esercizio. Assicurarsi di rispondere a tutte le richieste!

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

...

Domanda 5

Risposta non data

Non valutata

Si giustifichi la scelta del modello combinatorio adottato.

Domanda 6

Risposta non data

Non valutata

Si descrivano i criteri di pruning adottati o il motivo della loro assenza.

Informazione**PAGINA DI INTERMEZZO**

La prova da 12 punti termina prima di questa pagina di intermezzo.

La prossima domanda rappresenta l'inizio della prova da 18 punti.

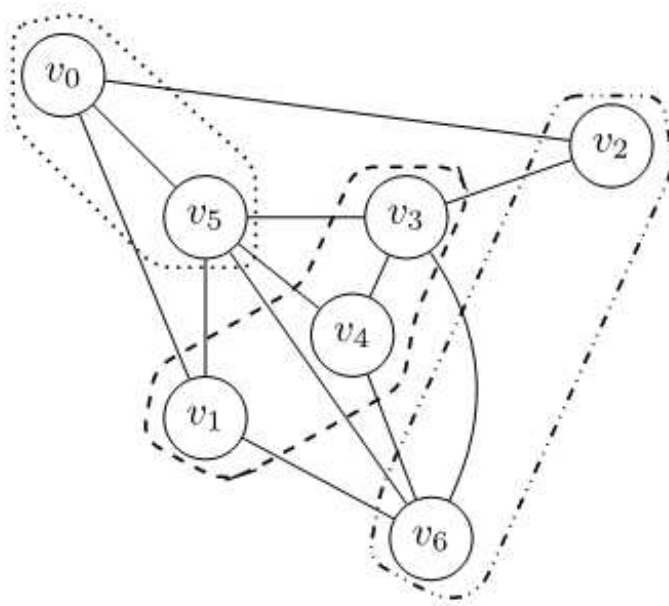
Informazione**Inizio prova da 18 punti**

Dato un grafo $G = (V, E)$ non orientato non pesato, viene definita **domatic partition** una partizione $\{V_1, V_2, \dots, V_k\}$ dei vertici del grafo tale per cui ogni sottoinsieme V_i è un **dominating set** per il grafo stesso.

Si definisce **dominating set** un sottoinsieme D dei vertici di un grafo tale per cui ogni vertice non in D è adiacente ad almeno un vertice di D .

Esempio:

Sia dato il seguente grafo,



la partizione di V $\{V_1 = \{v_0, v_5\}, V_2 = \{v_1, v_3, v_4\}, V_3 = \{v_2, v_6\}\}$ è una **domatic partition** in quanto soddisfa i criteri per essere una partizione ed ognuno dei sottoinsiemi che vi appartengono è un **dominating set**. La sua cardinalità è 3 ed è anche la cardinalità massima.

Si legga in una opportuna struttura dati un grafo G dal file `g.txt` che ha il seguente formato:

- sulla prima riga compare il numero di vertici V
- segue un numero non definito di coppie (v,w) a rappresentare gli archi del grafo, con $0 \leq v < V, 0 \leq w < V$

Si assuma che il grafo non sia un multigrafo e che sia semplice.

Si legga una proposta di partizionamento da file, in un formato a discrezione del candidato, e si valuti se questa sia una **domatic partition** valida, che quindi:

- rappresenti un partizionamento valido dei vertici
- sia tale che ogni sottoinsieme sia un dominating set

Si individui la **domatic partition** a cardinalità massima, ossia la partizione composta dal maggior numero possibile di dominating set per il grafo in input.

Domanda 7

Completo

Non valutata

Lettura

Scrivere qui la funzione di lettura del file g.txt.

Di seguito vengono elencate nuovamente le caratteristiche del file di input

- sulla prima riga compare il numero di vertici V
- segue un numero non definito di coppie (v,w) a rappresentare gli archi del grafo, con $0 \leq v < V$, $0 \leq w < V$

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

```
Graph GRAPHload(FILE *fin) {
    int dim, v, w;
    Graph g;

    fscanf(fin, "%d", &dim);
    g = GRAPHinit(dim);
    while (fscanf(fin, "%d %d", &v, &w) == 2) {
        GRAPHinsertE(g, v, w);
    }
    return g;
}
```

Domanda 8

Completo

Non valutata

Funzione di verifica

Scrivere in questa sezione la funzione di verifica della soluzione contenuta all'interno del file proposta.txt. Si ricorda che il formato del file è a discrezione del candidato.

Si valuti se questa sia una domatic partition valida, che quindi:

- rappresenti un partizionamento valido dei vertici
- sia tale che ogni sottinsieme sia un dominating set

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

```
//RAPPRESENTAZIONE FILE: prima riga indica il numero di partizioni, dalla seconda riga la prima colonna indica il numero di nodi presenti nella partizione, a seguire i nodi stessi seguiti da spazi (esempio come da consegna -- riga 1: 3, riga 2: 2 0 5 etc..)
```

```
int verify(FILE *fin, Graph G) {
    int n;
    int **m, *v; //Vettore v per le dimensioni delle partizioni lette da file, matrice m per i
    //salvataggio delle partizioni
    int *cnt = calloc(G->V, sizeof(int)); //Inizializzo vettore conteggio ricorrenze per partizionamenti
```

```
//LETTURA FILE
```

```
fscanf(fin, "%d", &n); //Numero di partizioni definite sulla prima riga del file
m = malloc(n*sizeof(*int));
v = calloc(n, sizeof(int));
for (int i = 0; i < n; i++) {
    fscanf(fin, "%d ", &v[i]);
    m[i] = malloc (v[i]*sizeof(int));
    for (int j = 0; j < v[i]; j++) {
        fscanf(fin, "%d ", &m[i][j]);
    }
}
```

```
//VERIFICA PARTIZIONAMENTO CORRETTO
```

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < v[i]; j++) {
        cnt[m[i][j]] += 1;
        if (cnt[m[i][j]] > 1) //Se ripetuto, no partizionamento, ritorno 0
            return 0;
    }
}
for (int i = 0; i < G->V; i++) {
    if (cnt[i] == 0)
        return 0; // Partizione non comprende tutti i vertici, ritorno 0
}
```

```
//VERIFICA DOMINATING SET
```

```
for (int i = 0; i < n; i++) {
    if(!DScheck(v[i], m[i], G))
        return 0; //Almeno uno dei sottoinsiemi non e' dominating set
```

```

    }
    return 1;
}

int DScheck (int c, int *v, Graph G) {
    int *visited = calloc(G->V, sizeof(int));
    for (int i = 0; i < c; i++) {
        for (int j = 0; j < G->V; j++) {
            if (G->adj[v[i]][j])
                visited[j] = 1;
        }
    }
    for (int i = 0; i < G->V; i++) {
        if (!visited[i])
            return 0; //Se almeno un vertice non e' stato visitato, ritorno 0
    }
    return 1;
}

```

Domanda 9

Completo

Non valutata

Ricorsione

Si individui la domatic partition a cardinalità massima, ossia la partizione composta dal maggior numero possibile di dominating set per il grafo in input.

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

// Inserire qui il codice